# DATA STRUCTURE *using* 'C'

## ADT (Abstract data type)

The definition of ADT only mentions *what operations are to be performed* but not how these operations will be implemented.

**Abstract Data Type** (ADT) is a **data type**, where *only behavior is defined* but ***not implementation.***

Opposite of ADT is **Concrete Data Type** (CDT), where it contains an ***implementation of ADT***.

*Examples:* ***Queue, Stack*** are ***ADTs***. Each of these ADTs has many implementations.

An **abstract data type** is a **type** with **associated** *operations,* but whose ***representation is hidden****.*

- **The stacks and queues** can be implemented by **arrays** or **linked lists**.
- A **heap** can be implemented by **an array** or **a binary tree.**

## Concrete data type (*Implementation)*

**A concrete data type (CDT)** is *a user-defined type* that ***behaves just like* a built-in type**.

A **concrete data type** is an opposite of an Abstract **data type**.

A **concrete data type** is rarely reusable beyond its original use, but can be embedded or composed with other **data types** to form larger **data types**.

***A concrete data type is absolutely defined***.
*Examples: Balanced Tree.*

**NOTE:** Creation of user define ***Class*** is ***Abstract*** data type and *Implementation of class* by using ***Object*** is ***Concrete*** data type.

## INTRODUCTION ABOUT DATA STRUCTURE

Data Structure is a ***"way of collecting and organizing data"*** in such a way that we **can perform operations on effective way**.

(or)

Data structure is a ***"Specific way to store and organize data in a computer's memory"***

(or)

***"Data may be arranged in many different ways such as the logical or mathematical model for a particular organization of data"*** is termed as a data structure.

(or)

Data structures that **"*stores the data in a special way so that* we can access and use the data efficiently".**

(or)

Data structure is **"*an arrangement of data in computer's memory in such a way that it* could make the data quickly available to the processor *for required purpose (such as calculations)"*.**

## Characteristics of a Data Structure

1. **Correctness**: Data structure implementation should implement its *interface correctly.*
2. **Time Complexity:** *Running time* or the *execution time* of operations of data structure must be as *small (little bit)* or *very less* as possible is called ***Best case***.

   The time complexity of an algorithm is the ***amount of time it takes*** for each statement to complete.
3. **Space Complexity:** *Memory usage* of a data structure operation should be as *little* as possible is called ***Best case***.

   ***Calculate the space*** occupied by ***variables***/***object*** in an algorithm/program to determine space complexity.

***Best case*** is the function which *performs the **minimum** number of steps* on input data of n elements.
***Worst case*** is the function which *performs the **maximum** number of steps* on input data of size n.
***Average case*** is the function which *performs an **average** (between min & max) number of steps* on input data of n elements.

**Popular Notations in Complexity Analysis of Algorithms**

### 1. Big-O Notation

We define an algorithm's **worst-case** time complexity by using the **Big-O** *notation*, which determines the set of functions grows ***slowe***r than or at the same rate as the expression. Furthermore, it explains *the maximum amount of time an algorithm requires* to consider all input values.

### 2. Omega Notation

It defines the **best case** of an algorithm's time complexity, the ***Omega notation*** defines whether the set of functions will grow ***faster*** or at the same rate as the expression. Furthermore, it explains the *minimum amount of time an algorithm requires* to consider all input values.

### 3. Theta Notation

It defines the **average case** of an algorithm's time complexity, the **Theta notation** defines when the set of functions lies in both Big - **O(expression)** and **Omega(expression)**, then Theta notation is used. This is how we define a time complexity average case for an algorithm.

**Advantages of Data Structure:**

- **Data Organization**:
  - We can ***access efficiently*** when we need that particular data.

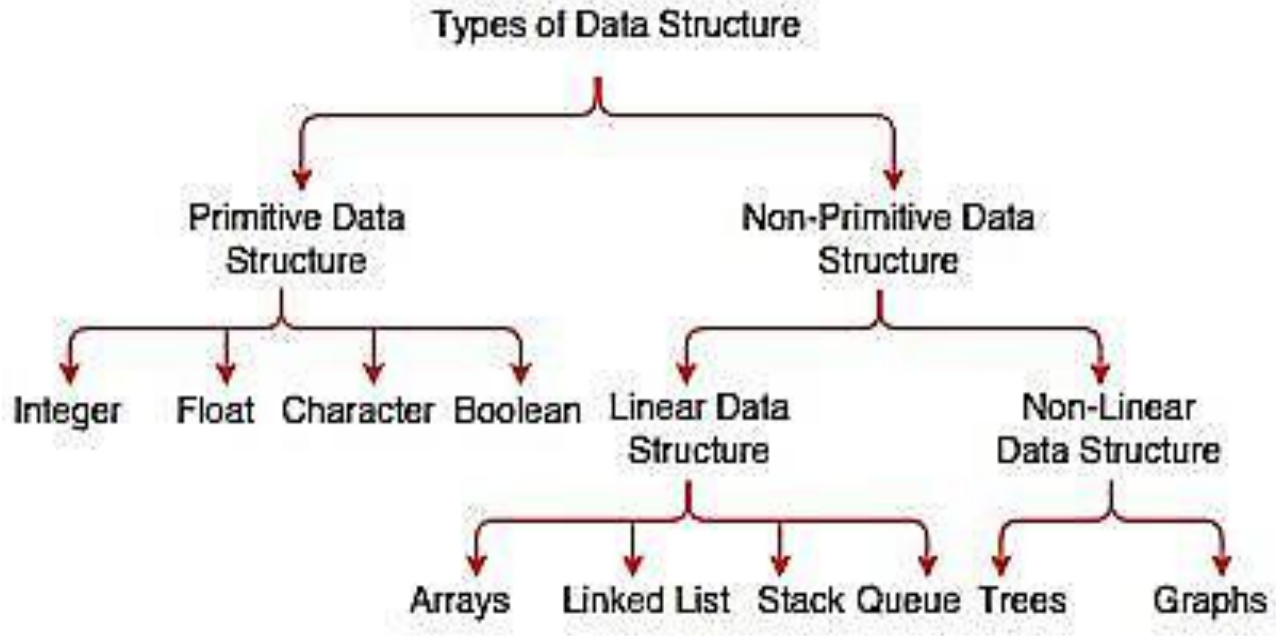
- **Efficiency**:
  - The main reason we organize the data is to ***improve the efficiency***.
  - When we need to perform several operations such as *add, delete update and search* on arrays, it takes more time in arrays than some of the other data structures. So the fact that we are interested in other data structures is because of the efficiency.

**Application of data structure:**

- ✓ ***DBMS*** and ***OS***
- ✓ **Compiler design**, ***Neural network*** and ***Networking structure***
- ✓ ***Statistical*** and ***Numerical analysis***
- ✓ ***Graphics***, ***AI*** and ***Simulation***

## Types of data structure

- ❖ ***Primitive data structure*** is a fundamental type of data structure that stores the data *of* ***only one type*** *by default (system).*
- ❖ The ***non-primitive data structure*** is a type of data structure which is a ***user-defined*** that *stores the data of different types in a single entity*.




The above listed are comes under the category of "*Primitive Data Structures"* (also known as built-in data structure or built-in data types).

**(ii) Non-primitive data structure**

- ✓ **Linear data structure:** The data items are arranged "*in a linear sequence"*.
  Example:
  - Array
  - Stack
  - Queue
  - Linked list (list)

- ✓ **Non-Linear data structure:** The data items are arranged "*not in a linear sequence".*

Example:

- Tree
- Graph

**Do you Know (***NOT in your Syllabus***)**

**What are Pointers?**

- It is *derived data type*.
- A ***pointer*** is **a variable** to *store* only ***address*** *of another variable* with *same data type*.
- The pointer variable *controls* (remote access) another data variable.

**Pointer Declaration**

- *Pointer declaration* is *data type with* ***variable*** but put (*asterisk*) * before (pointer) variable name.

***Declaration Syntax:***

***Datatype *variable_name;***

*Example:* int *p;

Here *p is a pointer variable.
NOTE: **&** *Address of* operator and
* is called ***dereference*** (or) *indirection operator*.

***Pointer Initialization:*** is the process of ***assigning address of a same type variable*** to a ***pointer*** variable.

Example:

```
#include<stdio.h>
main()
{
    int a=10, *b=&a;
    printf("%p",&a);
    printf("\n%X",b);
    *b=20;     //a=20
    printf("\n%d",a);
    printf("\n%d",*b);
    ++*b;  //++a;
    printf("\n%d",a);
}
```

```
000000000023FE44
23FE44
20
20
21
```

Compiler

**Memory Management**

*What is Memory Management?*

Memory management is a process of managing computer memory, assigning the memory space to the programs to improve the overall system performance.

As ***Static memory management (allocation)***

As we know that arrays store the homogeneous data, so most of the time, memory is allocated to the array at the declaration time is called static memory allocation.

Example: int a[10]; //static memory allocation

*Disadvantages*

i) Chances for Wastage of memory,

ii) Limited memory allocation.

As ***Dynamic memory management (allocation)***

In ***C language***, we use the **malloc()** or **calloc()** and **realloc()** functions to allocate the memory dynamically at run time, and ***free()*** function is used to deallocate the dynamically allocated memory using <stdlib.h>

In **C++** also supports these functions, but C++ also defines ***unary operators*** such as **new** and **delete** to perform the same tasks, i.e., ***allocating*** and ***freeing*** the ***memory dynamically***

The **'new'** operator to allocate the *memory dynamically* at the run time with help of pointer.

Example: int *a = new int[]; // in C++

*Advantages*

i) No chances for wastage of memory.

ii) Unlimited memory allocation.

## Self-referential Structure (***NOT in your Syllabus***)

Self-Referential structures are **those structures that have one or more pointers which point to the same type of structure, as their member (another).**

A self-referential structure is **a structure that can have members which point to a structure variable (another) of the same type**.

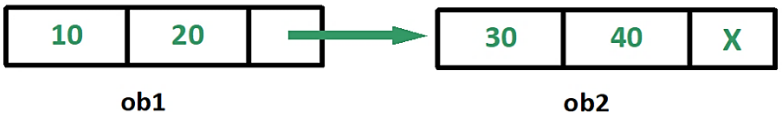
***Use of Self-referential structures***

Self- referential structures are very useful in applications that involve *linked data structures*, such as **lists** and **trees**.

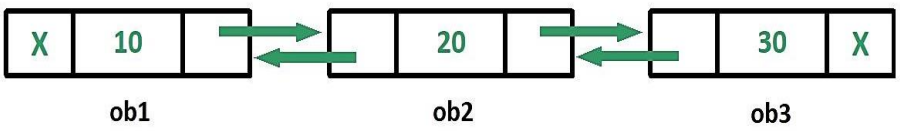
**Types of Self Referential Structures**

1. Self-Referential Structure with ***Single Link***
2. Self -Referential Structure with ***Multiple Links***

***1.Self-Referential Structure with* Single Link**




***2. Self -Referential Structure with* Multiple Links**



***Example1 using 'C'***

```
/*Self-Referential (single link) structures are those structures that have
  one or more pointers which point to
  the same type of structure, as their member (another) */
//POINTER OBJECT ONLY HAS OWN VALUE AND
//ADDRESS OF OTHER OBJECT (but not possible t in pointer variable)
#include<stdio.h>
#include<stdlib.h>
struct node  //User define data type and no memory of members
{
int m;  //member variable
struct node *next;   // member obj
}*a,*b; //structure variables/Objects pointer // With memory
main(){
        a=(struct node *) malloc( sizeof(struct node) );
        //DYNAMIC (HEAP) MEMORY DECL
        //printf("Address of obj A: %p",a); //address of 'A' obj
        printf("\nEnter Own DATA for obj A : ");
        scanf("%d",&a->m);   //AAAA  50
        b=(struct node *)malloc(sizeof(struct node) );
        printf("Enter Own DATA for obj B: ");
        scanf("%d",&b->m);   //bbbb  100
        b->next=a;  //printf("obj B contains: %p",b->next); //address of 'A' obj
printf("\nB own data: %d ",b->m);  // Data of 'B'
printf("\nB Ref data: %d ",b->next->m);  // Data of 'A'
b->next->m=999;
printf("\nA modified own data: %d ",a->m);
}
```

*Output*

```
Enter Own DATA for obj A : 10
Enter Own DATA for obj B: 20

B  own data: 20
B   Ref data: 10
A   modified own data: 999
```

*Example2 using **C++***

```
/*Self Referential (single link) structures are those structures that have  one or more pointers which point to
The same type of structure, as their member (another) */
#include<iostream>
#include<string.h>
using namespace std;
struct vijay
{
int m; char n[20];
struct vijay *p;
}a,b;
main()
{
cout<<"Enter name, mark for A: ";
cin>>a.n>>a.m;
cout<<"Enter name, mark for B: ";
cin>>b.n>>b.m;
b.p=&a;
cout<<"\nA : "<<a.n<<a.m;
cout<<"\nB : "<<b.n<<b.m;
cout<<"\nRef B: "<<b.p->n<<b.p->m;
b.p->m=100;
strcpy(a.n,"Apple");
cout<<endl<<a.m<<"   "<<b.p->n;    //Mutable
}
```

```
Enter name, mark for A: Vijay 60
Enter name, mark for B: Jagan 90

A : Vijay60
B : Jagan90
Ref B: Vijay60
100   Apple
```

***Note1 for* dot ( . ) *operator:***

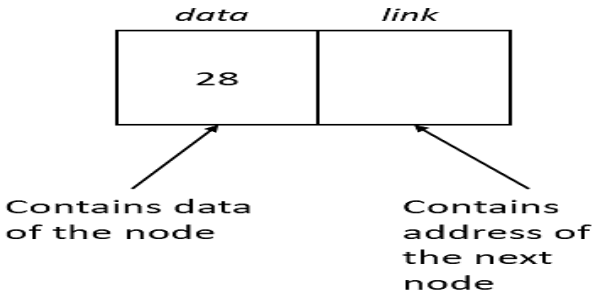
**The dot ( . ) is membership operator** *can be used to* ***link member variables of struct with static type struct objects (struct variables)***

***Note2 for* arrow ( -> ) *operator:***

**The arrow ( -> ) is membership operator** *can be used to* ***link member variables of struct with dynamic type struct object pointers (struct pointer variables)***

## Basic concepts of LINKED LIST

- ✓ A linked list is *a linear collection of data elements (or) nodes*
- ✓ Each nodes has
  - Data
  - Link or next or address or pointer




***Syntax***

```
struct node
        {
                int data;
                struct node *link;
        }*head;
```

**Advantage**

- ✓ Dynamic memory allocation

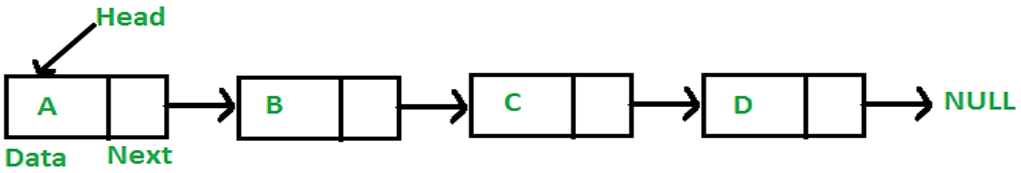
**Disadvantage**

- ✓ Need extra memory for address (Link)

***Explanation:*** The principal benefit of a linked list over a conventional array is that the list elements can be easily inserted or removed without reallocation or reorganization of the entire structure because the data items need not be stored contiguously in memory or on disk, while restructuring an array at run-time is a much more expensive operation.

**Dynamic data structure**: A linked list is a ***dynamic arrangement*** so it can ***grow and shrink at runtime by allocating and deallocating memory***.

So there is ***no need*** to give the ***initial size*** *of the linked list and also is* ***not overflow (full)***




*A linked list* consists of a data element known as **a node**.
And *each node consists* of two fields:

***First field*** has **data**, and
***Second field*** has **an address** *that keeps a reference to the* ***next node***.


Difference between **Array** and **Linked List**

| S.No. | ARRAY (*as Static*) | LINKED LIST (*as Dynamic*) |
|---|---|---|
| **1.** | An array is a *grouping of data elements* of equivalent data type. | A linked list is a *group of entities called a node*. The node includes two segments: data and address. |
| **2.** | It *stores* the data elements in a ***contiguous (continuous) memory zone.*** | It *stores elements* ***randomly***, or we can say anywhere in the memory zone. |
| **3.** | In the case of an array, ***memory size is fixed***, and it ***is not possible to change*** it *during the* ***run time****.* | In the linked list, the placement of elements is ***allocated during the run time***. |
| **4.** | The *elements are not dependent* on each other. So *stored in only* **Stack *memory*** | The data elements are dependent on each other. So stored in **Heap *memory*** *except head node element* |
| **5.** | The *memory is assigned* at ***compile time.*** | The *memory is assigned* at ***run time***. |
| **7.** | In the case of an array, *memory utilization is* ***ineffective****.* | In the case of the linked list, *memory utilization is* ***effective****.* |
| **8** | When it comes to executing any *operation like insertion, deletion*, array takes ***more time***. | When it comes to executing any *operation like insertion, deletion*, the linked list takes ***less time.*** |

## Application of Linked List

- Polynomial Manipulation representation
- Implementation for stack, Queue, tree, graph, Hash table for Dynamic
- Addition of long positive integers
- Representation of sparse matrices
- Addition of long positive integers
- Symbol table creation
- Mailing list

- Memory management
- Linked allocation of files
- Multiple precision arithmetic etc

## Types of Linked List

***What Are the Types of Linked Lists?***

- **Singly linked lists.**
- **Doubly linked lists.**
- **Circular linked lists.**
  - **Circular Singly linked lists**
  - **Circular doubly linked lists.**

## SINGLY LINKED LIST

A **singly linked list** is a type of linked list that is **unidirectional,** that is, it can be traversed in only one direction from head to the last node (tail). Each element in a linked list is called a **node**.

The nodes which have a data field as well as 'next' field, which points to the next node in line of nodes.

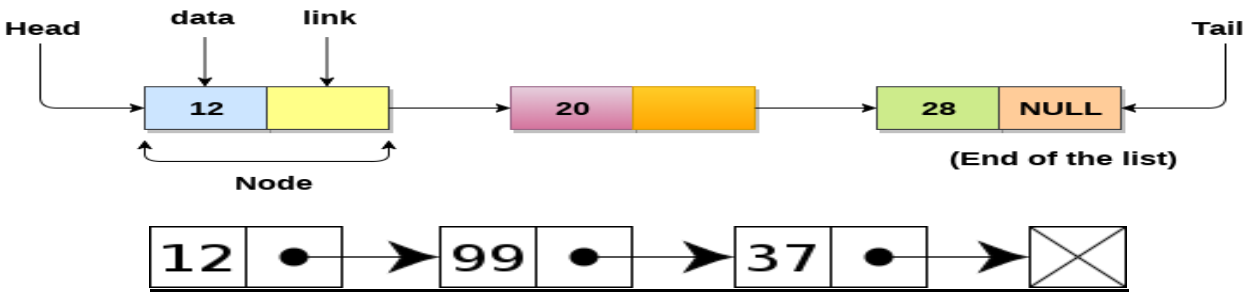
"***Item navigation(Direction) is forward only.***"

- The first node is called the **head**; it points to the first node of the list ***stored in*** **Stack** ***memory*** and helps us access every other element in the list
- The following nodes of head node to last node, also sometimes called the **tail** ***nodes stored in*** **Heap memory**, points to ***NULL*** which helps us in determining when the list ends (last node).

**Operations**

- ✓ Insertion-Preponed, Append/Postponed, Middle
- ✓ Deletion – Head, tail, Middle
- ✓ Traversal (only one direction – forward)
- ✓ Search

**Diagram**




**Creating Linked Stack Data Structure:**

- Allocate memory for each new element ***dynamically***

- Each node in the stack should contain *two parts*:
  - ***data***: the user's data
  - ***next***: the *address of the next element* in the stack

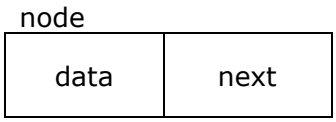
The Structure for Linked Stack is given as: (Using ***pointer*** and ***self-referential structure*** for *dynamic type memory allocation* and *link with address of next element*)

***Self-referential structures*** are mainly used **for the implementation of dynamic data structures like tree, linked list, etc**.

***Syntax***

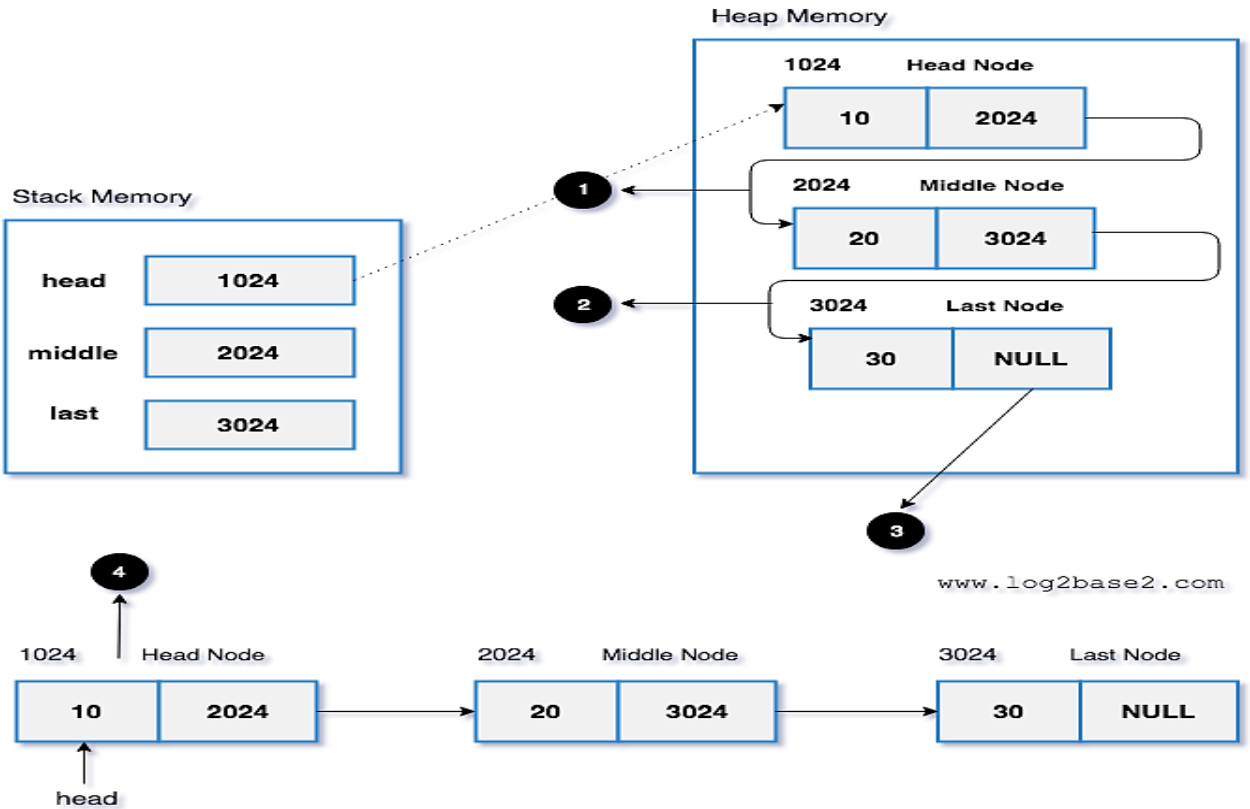
```
struct node
{
     int data;
     struct node *next;          // node *next;
}*top;
```

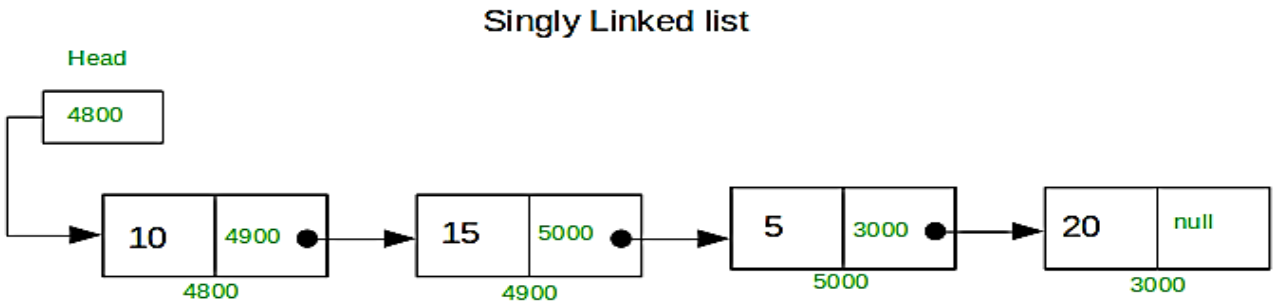
**Pictorial representation – node**




**Note:**

data = *any data type*, in our example, it is declared as integer
next = it is *pointer*, it is used to denote *address of next node*






*Example in C++ for Single node representation*

```
#include <iostream>
using namespace std;
struct node    //user define data type
{
  int data;
  node *next;
};
main()
{
   node *head = NULL;   //*ptr
   node *obj1=new node(); // Declaring 1st object for new node
       cout<<"1st new obj (node) address: "<<&*obj1<<endl;
      obj1->data = 10;   //1st input value
      obj1->next = head;  // address of head  //1st time is NULL.
    head = obj1; //  both data and address of new node assign to head
}
```

*Example Program in C++*

```
//Linked list with self referential struct
//with object pointer using new operator
#include <iostream>
using namespace std;
struct node    //user define data type
{
  int data;
  node *next;
};
      node *head = NULL;   //*ptr
   void insert(int value) {
    node *obj=new node(); // Declaring 1st object for new node
      obj->data = value; // cin>>a;  //1st input value  //10  //20
      obj->next = head; // address of head  //NULL  //0X.....10
    head = obj; //  both data and address of new node assign to head
   }
   void display() {
            node *ptr;
            ptr=head;
```

```
    while(ptr!=NULL) {
            cout<<ptr->data<<"  ";  //20
              ptr=ptr->next;  }
 }
 main()  {
         insert(10);
          insert(20);
          insert(40);
       cout<<"\nDisplay the linked list items: "<<endl;
     display();
      insert(111);
     insert(222);
    cout<<"\nDisplay the linked list items: "<<endl;
          display();     }
```

*Output*

```
Display the linked list items:
40  20  10
Display the linked list items:
222  111  40  20  10
--------------------------------
```

*Example Program in 'C' for ALL Operations*

```
//Singly Linked List Operations
//Prepond, Postpond/append,
//Insert, Delete, Display/Traverse Operations
#include <stdio.h>
#include <stdlib.h>
struct node{
  int  data;
  struct node *next;
}*head, *tail, *ptr, *n;  //*head for POSITION
//function to create a new node
void  createnode() {
  //allocate memory for the new node
  n = (struct node *) malloc( sizeof(struct node) );
   /*printf("\nnew 1st node address: %d",&*n);*/
  //accept input for new node data
  printf("\nEnter data for new node ");
  scanf("%d",&n->data);
}
//function to add node in the beginning
void  addbeg() { //Preponed
   createnode(); //function call to create a node
     if (head==NULL) {//list is empty
         n->next = NULL;   // for First node only
```

```
          tail = head = n; // for next node
  //printf("\nnow head node contains:%d",head->data);
         }
  else {
    n->next = head;    head = n;
              }
  printf("\n--->> New node inserted at the beginning of the list\n\n");
}
//function to add node at the end of the list
void  addend()     //Postponed /append
{
  createnode(); //function call to create a node
  if (head==NULL) { //list is empty //First time only
      n->next=NULL;
          tail = head = n; //new node is head node and tail node
       }
  else {
              n->next=NULL;
     tail->next = n; //stores address of new node into existing last node
     tail = n;  //
   }
       //n->next=NULL;
      printf("\n---<< New node inserted at the end of the list");
}
// Search a node
void search()
{
 int s, flag=0;
 if ( head == NULL )
 {
       printf("\nEmpty list.....\n");
       return;
 }
 printf("\nEnter search node data ");
 scanf("%d",&s);
 //locate the node containing search data s
 for(ptr=head; ptr!=NULL; ptr=ptr->next) //head->next
 {
   if (ptr->data == s)
   {
      flag=1; //search node located
      break;
   }
 }
 if (flag==0)
```

```
    printf("\n---> NO.Search node not located.....\n");
  else
        printf("\n---> YES.Search node is located ....\n");
}

//function to insert node in between
void insert()   //at middle
{
  int s, flag=0;
  if ( head == NULL )
  {
        printf("\n--->Empty list...Cannot insert...\n");
        return;
  }
  printf("\nEnter search node data ");
  scanf("%d",&s);
  //locate the node containing search data s
  for(ptr=head; ptr!=NULL; ptr=ptr->next)//head->next
  {
    if (ptr->data == s)
    {
       flag=1; //search node located
       break;
    }
  }
  if (flag==0)
    printf("\n--->>Search node not located...Cannot insert...\n");
  else
  {
    createnode(); //function call to create a new node
    n->next = ptr->next; //new node inserted AFTER search existing node
    ptr->next = n;
    printf("\n--->>Node inserted...\n");
  }
}
//function to delete node from the list containing data x
void delnode(int x)
{
  struct node *previous, *t;
  int flag=0;
  if (head==NULL)
  {
    printf("\nEmpty list...Cannot delete...\n");
    return;
  }
```

```
 for(ptr=previous=head; ptr!=NULL ; ptr=ptr->next)
 {
   if (ptr->data == x)
    {
      flag=1; //search node located
      break;
    }
   previous=ptr;
 }
 if (flag==0)
   printf("\nSearch node does not exist...Cannot delete...\n");
 else
 {
    if (ptr==head && ptr==tail)
     {
       printf("\nDeleting single node\n");
       free(ptr);
       head=tail=NULL;
     }
    else if (ptr==tail) //deleting tail node
     {
       printf("\nDeleting tail node\n");
       t=ptr; //store tail node address in pointer t
       tail=previous; //make previous node as tail node
       tail->next=NULL;
                free(t); //free memory
    }
    else if (ptr==head)
     {
       printf("\nDeleting head node\n");
       t=ptr;
       head = head->next;
       free(t);
     }
    else    // delete middle node
     {
        printf("\nDeleting node in between \n");
        t=ptr;
        previous->next = ptr->next;
        free(t);
     }
     printf("\n--->Node deleted...\n");
  }
}
//function to display data in all the nodes
```

```
void  display()    //Traverse
{
   if (head==NULL)
   {
      printf("\nEmpty list\n");
       }
   else
   {
          for(ptr=head; ptr!=NULL; ptr=ptr->next) //ptr=head->next
      printf(" %d ",ptr->data);
  }
}
/* main() function */
main()
{
   int choice, d;
   head = tail = NULL;   //'\0'
   do
   {
      printf("\n1.Add node in the beginning\n");
      printf("\n2.Add node at the end\n");
      printf("\n3.Insert node\n");
      printf("\n4.Delete node\n");
      printf("\n5.Search node\n");
      printf("\n6.Display data in all the nodes\n");
      printf("\n7.Exit \n");
      printf("Enter your choice ");
      scanf("%d",&choice);
      switch( choice )
      {
         case 1:
               addbeg();
               break;
         case 2:
               addend();
               break;
         case 3:
               insert();
               break;
         case 4:
               printf("Enter data of node to delete ");
               scanf("%d",&d);
               delnode(d);
               break;
         case 5:
```

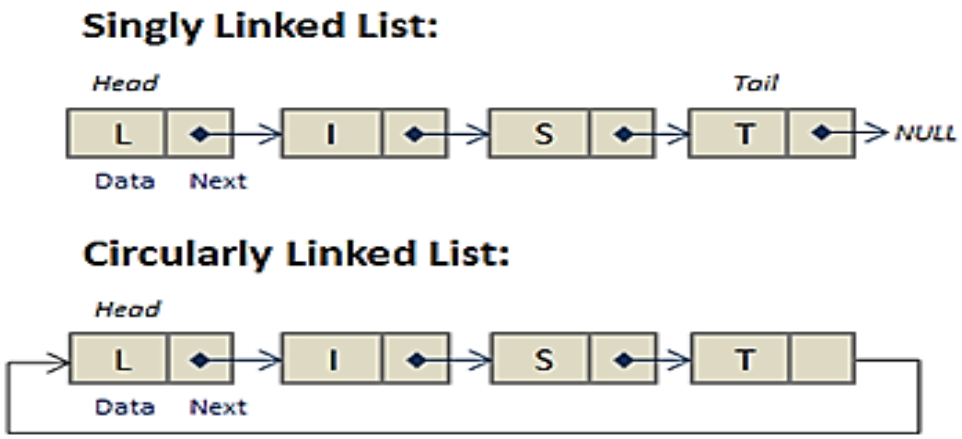
```
                search();
                break;
        case 6:
                display();
                break;
        case 7:
                exit(0);
                break;
        default:
                printf("\nInvalid choice\n");
     }
    }while (1);
}
```

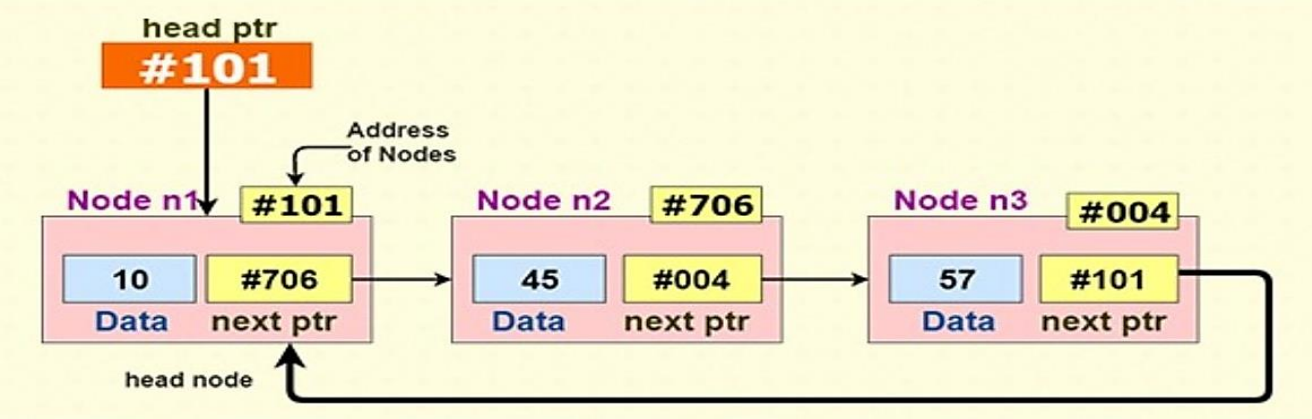
*Output*

```
1.Add node in the beginning
2.Add node at the end
3.Insert node
4.Delete node
5.Display data in all the nodes
6.Exit
Enter your choice 1
Enter data for new node 10
New node added at the beginning of the list
Enter your choice 1
Enter data for new node 20
New node added at the beginning of the list
Enter your choice 2
Enter data for new node 30
New node added at the end of the list
Enter your choice 5
 20   10   30
```

## Circular Linked List

- A *circular linked list* is a type of linked list in which **the first** and **the last nodes are also connected to each other to form a circle**.
- Here, the ***address of the last node consists of the address of the first node***.
- Both *Singly Linked List* and *Doubly Linked List* can be made into a *circular linked list.*




**NOTE**

- ✓ We *traverse a* ***circular singly linked list*** until we ***reach*** **the** ***same node where we started***.
- ✓ The ***circular singly liked list*** has ***no beginning*** and ***no ending***.
- ✓ There is ***no null value*** present in the *next part of any of the nodes*.

**Difference between Singly Linked List and Circular Linked List**

- o A *singly linked list* has a **NULL pointer** at the ***end of the list***.
- o But *a circular linked list* has a pointer that points **back to the first node address** in the list (at end of the list).

# Circular Lists

- Circular double-linked list:
  - Link last node to the first node, and
  - Link first node to the last node
- We can also build singly-linked circular lists:
  - Traverse in forward direction only
- **Advantages:**
  - Continue to traverse even after passing the first or last node
  - Visit all elements from any starting point
  - Never fall off the end of a list
- **Disadvantage:** Code must avoid an infinite loop!

**Application of Circular Linked List**

- Circular linked list are mostly used in ***task maintenance in operating systems***.
- It is also used by the *Operating system* to ***share time for different users***, generally uses a ***Round-Robin time-sharing mechanism***.
- Circular linked list are being used in computer science including ***browser surfing*** (**Browser cache** )where a record of pages ***visited in the past*** by the user, is ***maintained in the form of circular linked lists*** and ***can be accessed again on clicking the previous button***.
- ***Multiplayer games*** use a *circular list to swap between players* in a loop.
- ***Shopping-cart*** (A shopping cart on an *online retailer's site* that facilitates the purchase of a product or service) on ***online websites.***
- ***Managing songs playlist*** in *media player applications*.
- It can be used to ***Task Scheduling*** and ***Navigation systems***.
- It can be also used to ***Implementation of a Queue*** and ***hash table***.

**Operations on Circular Singly linked list:**

**Insertion**

| SN | Operation | Description |
|---|---|---|
| 1 | **Insertion at beginning** | **Adding a node into circular singly linked list at the beginning.** |

| | | |
|---|---|---|
| **2** | **Insertion at the end** | **Adding a node into circular singly linked list at the end.** |

## Deletion & Traversing

| SN | Operation | Description |
|---|---|---|
| **1** | **Deletion at beginning** | **Removing the node from circular singly linked list at the beginning.** |
| **2** | **Deletion at the end** | **Removing the node from circular singly linked list at the end.** |
| **3** | **Traversing** | **Visiting each element of the list at least once in order to perform some specific operation.** |

*Example Program in 'C' for All operations*

```
/* Circular singly linked list */
#include <stdio.h>
#include <stdlib.h>
struct node {
   int  data;
   struct node *next;
} *head, *tail, *ptr, *n;
//function to create a new node
void  createnode() {
  //allocate memory for the new node
  n = (struct node *) malloc( sizeof(struct node) );
  //accept input for new node data
  printf("\nEnter data for new node ");
  scanf("%d",&n->data);
}
//function to add node in the beginning
void  addbeg() {     //Prepond operation
```

```
    createnode(); //function call to create a node
    if (head==NULL) //list is empty //very 1st node creation only
    {   //Single node    //One time and First time
        tail = head = n; //new node is head node and tail node
        n->next = head; //=tail; //=n; //circular list
    }
    else  //From 2nd node to n node
    {
        n->next = head;
        head = n;
        tail->next=head;  //circular list //here tail is 1st node
    }
    printf("\nNew node added at the beginning of the list");
}
//function to add node at the end of the list
void  addend() {  //Postpond (or) Append
    createnode(); //function call to create a node
     if (head==NULL) //list is empty  //1st node creation
    {
        tail = head = n; //new node is head node and tail node
        n->next=head;   //circular list
    }
    else    //from 2nd node creation to n node
    {
        tail->next = n;
        tail = n;
        tail->next = head;  // Circular list
    }
    printf("\nNew node added at the end of the list");
```

```
}
void search() {   //same as Singly Linked List
  int s, flag=0;
  if (head==NULL) {
     printf("\nEmpty list\n");
     return;  }
  printf("\nEnter search node data ");
  scanf("%d",&s);
  //locate the node containing search data s
  ptr=head;
  do   {
     if (ptr->data == s) {
         flag=1; //search node located
         break;
        }
  ptr=ptr->next;
  }while(ptr!=head);
  if (flag==0)
     printf("\nNo. Search node is not located...\n");
  else
           printf("\nYes. Search node is located...\n");
}
//function to insert node in between
void insert() { //same as Singly Linked List
  int s, flag=0;
  if (head==NULL) {
     printf("\nEmpty list\n");
     return;  }
  printf("\nEnter search node data ");
```

```
  scanf("%d",&s);
  //locate the node containing search data s
  ptr=head;
  do   {
     if (ptr->data == s) {
         flag=1; //search node located
         break;
        }
  ptr=ptr->next;
  }while(ptr!=head);
  if (flag==0)
     printf("\nSearch node not located...\n");
  else  {
      createnode(); //function call to create a new node
      n->next = ptr->next;
      ptr->next = n;
      printf("\nNode inserted...\n");
  }
}
//function to delete node from the list containing data x
void delnode(int x)
{
 struct node *previous, *t;
  int flag=0;
  if (head==NULL)
  {
      printf("\nEmpty list...\n");
      return;
  }
```

```
ptr=previous=head;
do
{
  if (ptr->data == x)
   {
     flag=1; //search node located
     break;
   }
  previous=ptr;
  ptr=ptr->next;
}while(ptr!=head);
if (flag==0)
  printf("\nSearch node does not exist...\n");
else
 {
    if (ptr==head && ptr==tail)
    {
        free(ptr);
        head=tail=NULL;
    }
    else if (ptr==tail) //deleting tail node
     {
         t=tail;//=ptr; //store tail node address in pointer t
         tail=previous; //make previous node as tail node
         tail->next = head; //for Circular
         free(t); //free memory
     }
    else if (ptr==head)
    {
```

```
            t=ptr; //t=head;
            head = head->next;
            tail->next = head; //for Circular only
            free(t);
        }
        else     //delete middle node
        {
            t=ptr;
            previous->next = ptr->next;
            free(t);
        }
        printf("\nNode deleted...\n");
    }
}
//function to display data in all the nodes
void  display()
{
    if (head==NULL)
    {
        printf("\nEmpty list...\n");
        return;
    }
    ptr=head;
    do
    {
        printf(" %d  ",ptr->data);
        ptr=ptr->next;
    }while(ptr!=head);
    }
```

```
/* main() function */
 main()
{
   int choice, d;
   head = tail = NULL;
   do
   {
      printf("\n1.Add node in the beginning\n");
      printf("\n2.Add node at the end\n");
      printf("\n3.Insert node\n");
      printf("\n4.Delete node\n");
      printf("\n5.Display data in all the nodes\n");
      printf("\n6.Exit \n");
      printf("Enter your choice ");
      scanf("%d",&choice);
      switch( choice )
      {
         case 1:
               addbeg();
               break;
         case 2:
               addend();
               break;
         case 3:
               insert();
               break;
         case 4:
               printf("Enter data of node to delete ");
               scanf("%d",&d);
```

```
                delnode(d);
                break;
            case 5:
                display();
                break;
            case 6:
                exit(0);
                break;
            default:
                printf("\nInvalid choice...\n");
        }
         }while (1);
    }
```

*Output*

1.Add node in the beginning

2.Add node at the end

3.Insert node

4.Delete node

5.Display data in all the nodes

6.Exit

Enter your choice 1

Enter data for new node **10**

New node added at the beginning of the list

Enter your choice 1

Enter data for new node **20**

New node added at the beginning of the list

Enter your choice 2

Enter data for new node **30**

New node added at the end of the list

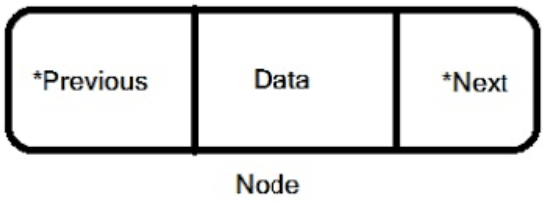
Enter your choice 5

***20 10 30***

## Doubly Linked List

- ✓ A doubly linked list (or) ***a two way linked list*** *(or)* ***Bidirectional linked list.***
- ✓ It is a type of linked list which contains a pointer to the ***next*** as well as ***previous*** node in the sequence.
- ✓ It consists of ***three parts***.
  - **Data**
  - **Pointer to the next node (forward)**
  - **Pointer to the previous node (backward)**
- ✓ ***"Items can be navigated forward and backward."***

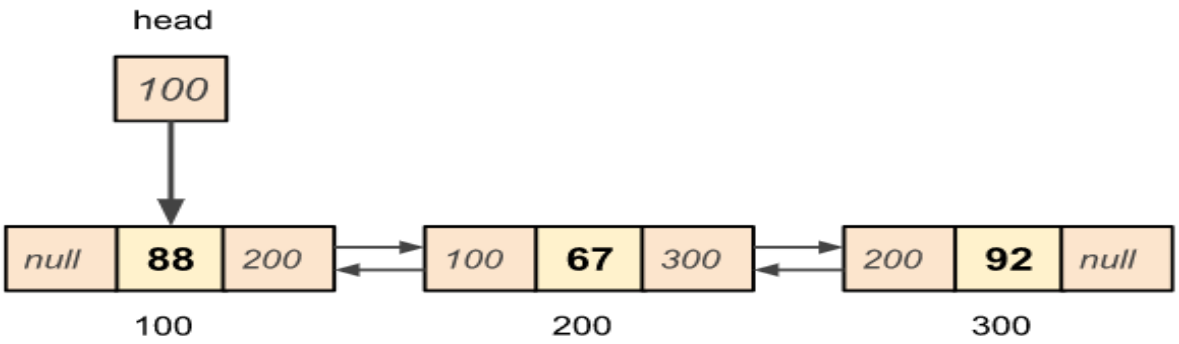
```
struct node
{
        struct node *prev;
        int data;
        struct node *next;
};
```

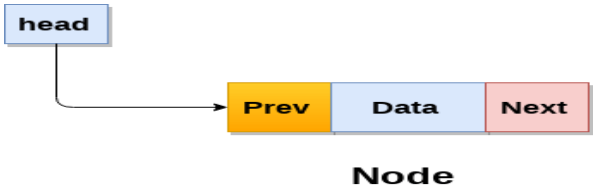


Node

**Basic Operation**

- **Insertion at Front** – Adds an element at the beginning of the list.
- **Deletion at Front**– deletes an element at the beginning of the list.
- **Insert Last** – Adds an element at the end of the list.
- **Delete Last** – Deletes an element from the end of the list.
- **Insert After** – Adds an element after an item of the list.
- **Delete** – Deletes an element from the list using the key.
- **Display** – Displays the complete list in a forward manner.

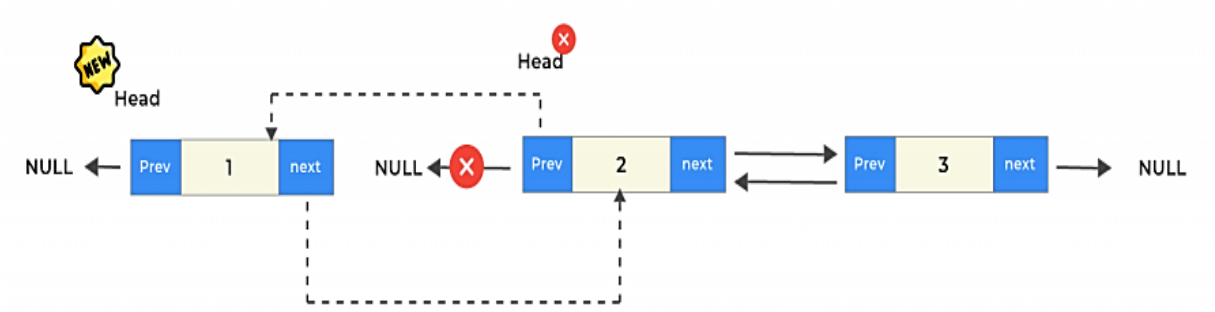

As per the above illustration, following are the important points to be considered.

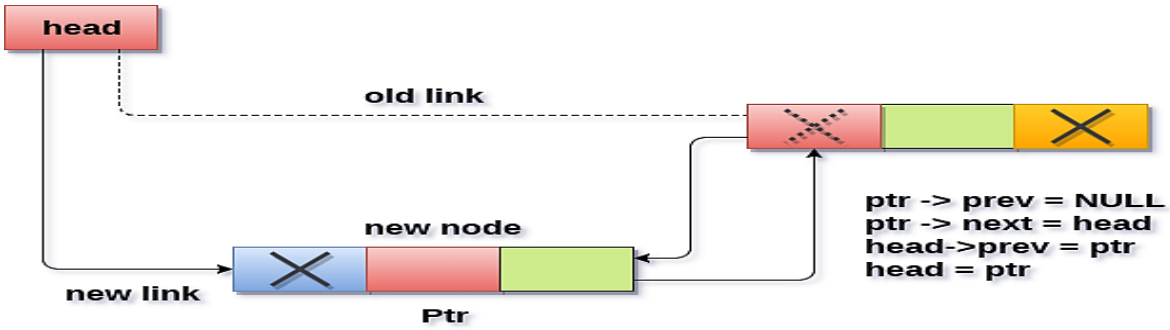
- *Doubly Linked List* contains *a link element (node) called* ***first*** *and* ***last****.*
- Each link(node) carries **a data field** and ***two link fields*** called ***next*** and ***prev***.
- Each link(node) is *linked with its* ***next link*** using its *next link*.
- Each link(node) is *also linked with its* ***previous link*** using its *previous link*.
- The *last link carries a link* as ***null*** *to mark the end of the list*.

An each node has ***3 components*** –

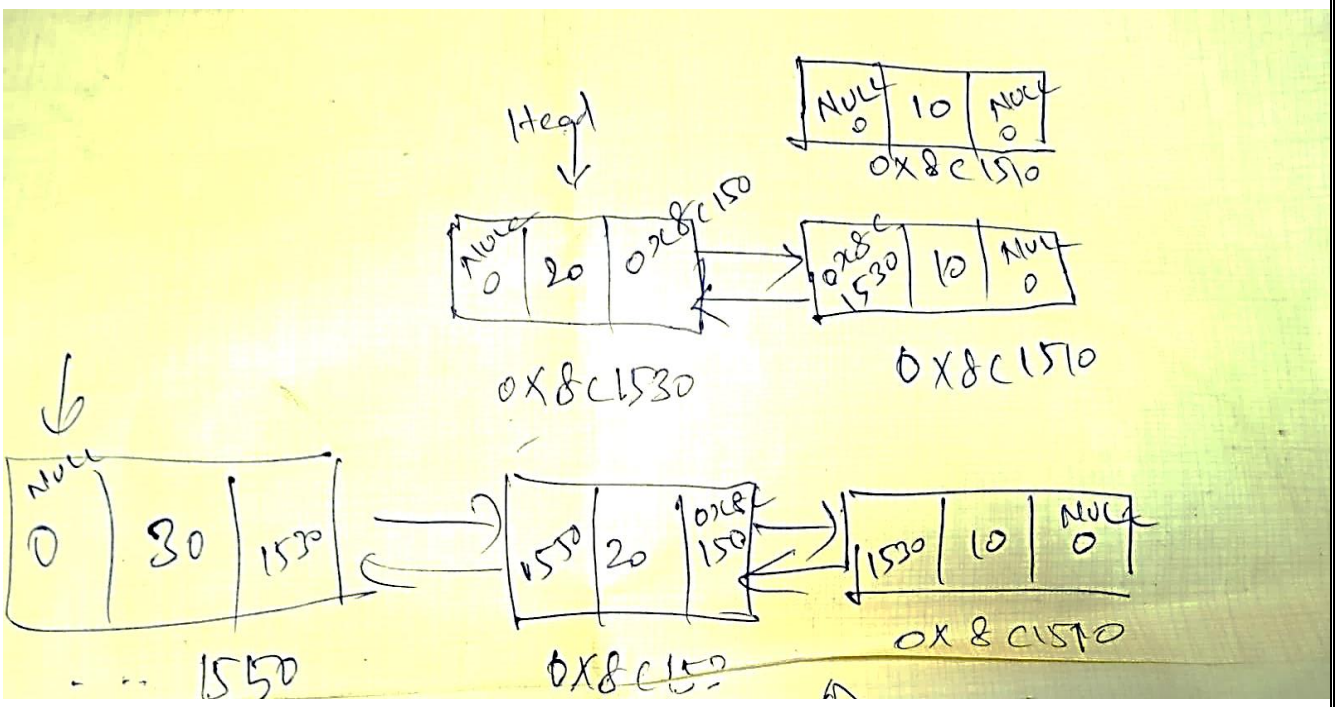
1. ***Data***
2. ***Previous Pointer***
3. ***Next Pointer***

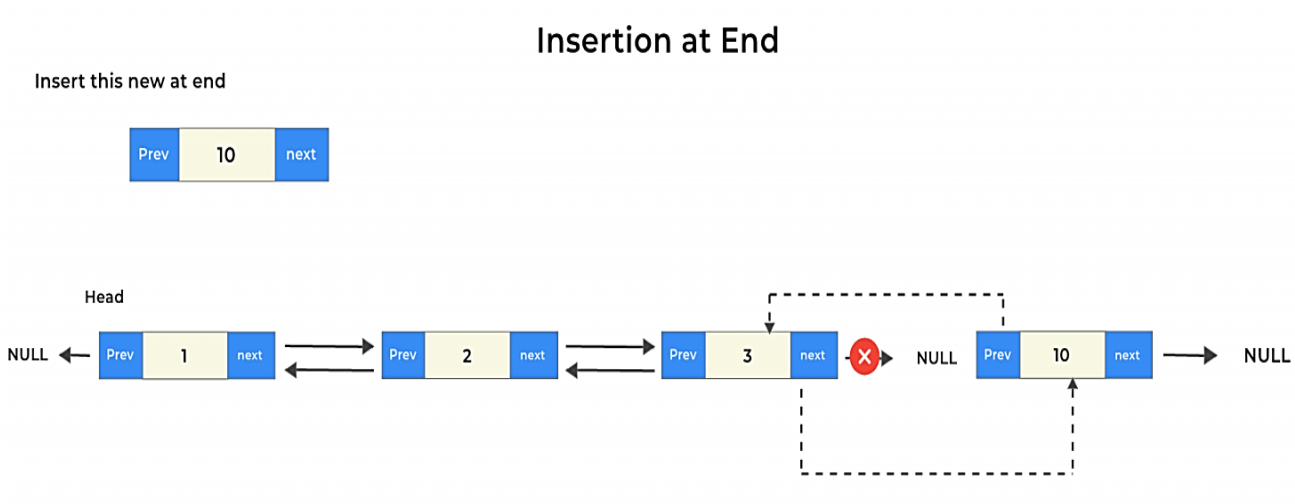
## Insertion at Beginning




**Insertion into doubly linked list at beginning**






***Example for Doubly Linked List for All Operations***

```
//Doubly Linked List Operatoions
#include <stdio.h>
#include <stdlib.h>
struct node{
  int  data;
  struct node *prev;
  struct node *next;
}*head, *tail, *ptr, *n;  //*head for POSITION
//function to create a new node
void  createnode() {
  //allocate memory for the new node
  n = (struct node *) malloc( sizeof(struct node) );
    //accept input for new node data
  printf("\nEnter data for new node ");
  scanf("%d",&n->data);
}
//function to add node in the beginning
void  addbeg() { //preponed
   createnode(); //function call to create a node
      if (head==NULL) {//list is empty
         n->prev = NULL;  // for First node only
                  n->next = NULL;
           tail = head = n; // object copy
```

```
            }
   else {{
            n->prev = NULL;}
      n->next = head;
            head->prev=n;
            head = n;}
   printf("\nNew node inserted at the beginning of the list\n\n");
}
//function to add node at the end of the list
void  addend()  //void append  //void postponed
{
   createnode(); //function call to create a node
   if (head==NULL) { //list is empty
        n->prev = NULL;  // for First node only
                  n->next=NULL;
                  tail = head = n; //new node is head node and tail node
    }
   else {
      tail->next = n; //stores address of new node into existing last node
      n->prev=tail;
            n->next=NULL;
            tail = n;
          }
  printf("\nNew node inserted at the end of the list");
}
//function to insert node in between
void insert()
{
  int s, flag=0;
  if ( head == NULL )
  {
          printf("\nEmpty list...Cannot insert...\n");
          return;
```

```
  }
  printf("\nEnter search node data to be inserted:  ");
  scanf("%d",&s);
  //locate the node containing search data s
  for(ptr=head; ptr!=NULL; ptr=ptr->next)//head->next
  {
     if (ptr->data == s)
     {
        flag=1; //search node located
        break;
     }
  }
  if (flag==0)
     printf("\nSearch node not located...Cannot insert...\n");
  else
  {
     createnode(); //function call to create a new node
     if(ptr!=tail){
              n->prev=ptr;
              n->next=ptr->next;
              ptr->next->prev=n;
              ptr->next=n;}
           else if(ptr==tail)
                    {tail->next = n; //Same code of Append/Postponed
         n->prev=tail;
               n->next=NULL;
               tail = n;
           }
  }
}
//function to delete node from the list containing data x
void delnode()
{
  struct node *t;
```

```
int x;
int flag=0;
printf("Enter data to be deleted!");
scanf("%d",&x);
if (head==NULL)
{
   printf("\nEmpty list...Cannot delete...\n");
   return;
}
for(ptr=head; ptr!=NULL ; ptr=ptr->next)
{
  if (ptr->data == x)
   {
     flag=1; //search node located
     break;
   }
  }
if (flag==0)
  printf("\nSearch node does not exist...Cannot delete...\n");
else
{
   if (ptr==head && ptr==tail) //Single node
    {
      printf("\nDeleting single node\n");
      free(ptr);
      head=tail=NULL;
      printf("\n**Empty List**");
    }
   else if (ptr==tail) //deleting tail node
    {
      printf("\nDeleting tail node\n");
      t=ptr; //store tail node address in pointer t
      //make previous node as tail node
```

```
        tail=tail->prev;  //change to doubly
                    free(t); //free memory
        tail->next=NULL;
      }
     else if (ptr==head)
     {
       printf("\nDeleting head node\n");
       t=ptr;
       head = head->next;
       head->prev=NULL; //doubly
       free(t);
     }
     else  //delete middle node
     {
        printf("\nDeleting middle node in the list\n");
        t=ptr;
        ptr->prev->next=ptr->next;
        ptr->next->prev=ptr->prev;
                    free(t);
     }
     printf("\nNode deleted...\n");
   }
}
//function to display data in all the nodes
void  display()
{
   if (head==NULL)
   {
      printf("\nEmpty list\n");
       }
   else
   {
        printf("\nFORWARD DIRECTION-->\n");
```

```
        for(ptr=head; ptr!=NULL; ptr=ptr->next)
      printf("%d  ",ptr->data);

        printf("\nBACKWARD DIRECTION <--\n");
   for(ptr=tail; ptr!=NULL; ptr=ptr->prev)
      printf("%d  ",ptr->data);
  }
}
/* main() function */
 main()
{
   int choice, d;
   head = tail = NULL;   //'\0'
   do
   {
      printf("\n1.Add node in the beginning\n");
      printf("\n2.Add node at the end\n");
      printf("\n3.Insert node\n");
      printf("\n4.Delete node\n");
      printf("\n5.Display data in all the nodes\n");
      printf("\n6.Exit \n");
      printf("Enter your choice ");
      scanf("%d",&choice);
      switch( choice )
      {
         case 1:
               addbeg();
               break;
         case 2:
               addend();
               break;
         case 3:
               insert();
               break;
```

```
            case 4:
                  delnode();
                  break;
            case 5:
                  display();
                  break;
            case 6:
                  exit(0);
                  break;
            default:
                  printf("\nInvalid choice\n");
       }
      }while (1);
}
```

*Output*

1.Add node in the beginning

2.Add node at the end

3.Insert node

4.Delete node

5.Display data in all the nodes

6.Exit
Enter your choice **1**

Enter data for new node **10**

New node inserted at the *beginning* of the list
Enter your choice **1**

Enter data for new node **20**

New node inserted at *the beginning* of the list
Enter your choice **2**

Enter data for new node **30**

New node inserted at *the **end** of the list*
Enter your choice **5**

*Traverse Forward:* 20 10 30
*Traverse Backward*: 30 10 20

**Advantages of a Doubly Linked List**

- Allows us to ***iterate in both directions***.
- We can *delete a node easily* as we have access to its previous node.
- *Reversing is easy*.
- Can *grow or shrink in size dynamically*.
- Useful in *implementing various other data structures*.

**Applications of Doubly Linked List**

- Implement ***undo-redo*** features,
- Great use of the doubly linked list is in ***navigation systems***, as it needs ***front and back navigation***.
- ***In Web browsers*** use doubly linked lists to ***store the browsing history of users***. Each page that is visited is added to the list, and users can ***move backward and forward*** through their browsing *history by traversing the list in both directions.*
- Again, it is easily possible to ***implement other data structures like a binary tree, hash tables, stack***, etc.
- It is used ***in music playing system*** where you can easily ***play the previous one or next one song as many times one person wants to***. Basically it provides full flexibility to perform functions and make the system user-friendly.
- In ***many operating systems***, the ***thread scheduler*** (the thing that chooses what processes need to run at which times) maintains a doubly-linked list of all the processes running at any time. This makes it easy to move a process from one queue (say, the list of active processes that need a turn to run) into another queue (say, the list of processes that are blocked and waiting for something to release them).
- It is used in a *famous game* concept which is a ***deck of cards***.
- Applications that have a ***Most Recently*** used (MRU) ***list*** (a linked list of file names)